

# Tutorial de implementacion paso a paso

---

## Wavetables y ADSR asistidas por Gemini para un sintetizador procedural

Guia de laboratorio para construir la funcionalidad desde el sintetizador base.

El estudiante parte de Osc.cs y escribe GeminiTimbreGenerator.cs desde cero.

# Tabla de contenido

- 1 Objetivos y alcance
- 2 Prerrequisitos
- 3 Arquitectura final
- 4 Obtener una API key de Gemini desde cero
- 5 Crear GeminiTimbreGenerator.cs desde cero
- 6 Estructuras de datos y configuracion
- 7 Conexion con Gemini y structured output
- 8 Parsing validacion y fallback
- 9 Construccion local de wavetable
- 10 ADSR y GeneratedTimbre
- 11 Modificar Osc.cs
- 12 Primera prueba con prompt
- 13 Analisis DSP de AudioClip
- 14 Gemini como complemento del audio
- 15 Segunda prueba con AudioClip
- 16 Interfaz minima en Unity
- 17 Audio thread
- 18Codigo completo de referencia
- 19 Actividad final
- 20 Referencias oficiales

# 1 Objetivos y alcance

Esta practica ensena a construir una capa de generacion de timbre que usa IA solo para producir descriptores estructurados. El audio final se genera localmente con el sintetizador procedural existente.

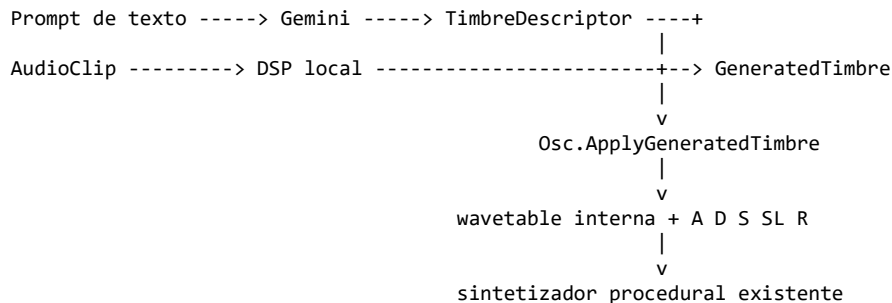
- Conectar Unity con Gemini API mediante UnityWebRequest.
- Usar structured output para obtener armonicos, timbre y ADSR.
- Convertir descriptores en una wavetable local de 2048 muestras.
- Analizar AudioClip localmente para extraer wavetable y ADSR.
- Aplicar el resultado a Osc.cs sin crear un segundo sintetizador.
- Configurar una UI minima para probar prompt y AudioClip.

## 2 Prerrequisitos

- Un sintetizador procedural con Osc.cs funcionando.
- TextMesh Pro disponible para TMP\_InputField y TextMeshProUGUI.
- Acceso a Internet para la modalidad por prompt.
- Una cuenta Google personal o institucional con permiso para Google AI Studio.
- Conocimientos basicos de componentes, Inspector y eventos OnClick en Unity.

## 3 Arquitectura final

La arquitectura separa UI, analisis, generacion de datos y render de audio. El hilo de audio solo consume datos listos.



## 4 Obtener una API key de Gemini desde cero

La documentacion oficial actual indica que las claves se gestionan desde Google AI Studio. Cada clave se asocia a un proyecto de Google Cloud. Para usuarios nuevos, AI Studio puede crear automaticamente un proyecto y una API key despues de aceptar los terminos.

### 4.1 Acceder al servicio oficial

1. Abre <https://aistudio.google.com/app/apikey>.
2. Si la pagina redirige, confirma que sigues en Google AI Studio y en la seccion API Keys.
3. Tambien puedes entrar por <https://ai.google.dev/gemini-api/docs/api-key> y usar el enlace a Google AI Studio.

### 4.2 Iniciar sesion

4. Inicia sesion con tu propia cuenta Google.
5. Puedes intentar una cuenta institucional, pero no asumas que todas tienen habilitado Gemini API.
6. Si la organizacion bloquea el acceso, usa una cuenta permitida o solicita permisos al administrador.

### 4.3 Crear u obtener la API key

7. En Google AI Studio abre Dashboard y entra a API Keys si no estas alli.
8. Si eres usuario nuevo, revisa si ya existe una clave creada por defecto.
9. Para crear otra, pulsa Create API key.
10. Selecciona o crea el proyecto que usaras cuando la interfaz lo solicite.
11. Cuando aparezca la clave, copiala. En esta practica local la asignaras en el campo geminiApiKey del Inspector de Unity.

Si Create API key no esta disponible, faltan permisos IAM. La documentacion oficial menciona permisos como `apikeykeys.create`, `serviceusage.services.enable`, `iam.serviceAccounts.create` e `iam.serviceAccountApiKeyBindings.create`.

### 4.4 Seguridad

La clave autoriza llamadas contra tu cuenta o proyecto. En esta practica local se asignara desde el Inspector para evitar escribirla en el archivo de codigo.

```
public string geminiApiKey = string.Empty;
```

No uses una clave de ejemplo ni la clave de otra persona. Pega tu propia clave solo en el Inspector de tu escena local y evita compartir capturas o archivos de escena que la muestren.

### 4.5 Asignar la API key desde el Inspector

En GeminiTimbreGenerator.cs agrega el campo publico geminiApiKey en la seccion Gemini. El codigo no contiene ninguna clave real:

```
[Header("Gemini")]  
[Tooltip("Assign each student's Gemini API key in the Inspector for local testing. Do not commit scenes or prefabs that contain real keys.")]  
public string geminiApiKey = string.Empty;  
public string modelName = "gemini-3.8-flash";
```

Guarda el script y vuelve a Unity para que compile. Luego selecciona el GameObject que tiene GeminiTimbreGenerator y pega tu clave real en el campo geminiApiKey del Inspector.

Para verificar sin imprimir la clave completa, el codigo solo comprueba si el campo esta vacio:

```
if (string.IsNullOrEmpty(geminiApiKey))  
    Debug.LogWarning("Configura geminiApiKey antes de llamar a Gemini.");
```

## Checkpoint 1 API key obtenida y configurada

### Que debe funcionar

Existe una API key propia y esta asignada en el campo geminiApiKey del Inspector.

### Como probarlo

Revisa el campo geminiApiKey en el Inspector y confirma que no esta vacio.

### Resultado esperado

El campo debe contener tu clave real.

### Errores frecuentes

- El campo geminiApiKey esta vacio: pega tu clave real en el Inspector.
- Si la cuenta institucional no permite AI Studio: usa otra cuenta permitida o pide permisos.
- La key se copio con espacios: elimina espacios antes o despues de la clave.

# 5 Crear GeminiTimbreGenerator.cs desde cero

- 12. En Unity localiza la carpeta de scripts del sintetizador.
- 13. Crea una carpeta TimbreAI si no existe.
- 14. Dentro de TimbreAI crea un C# Script llamado GeminiTimbreGenerator.
- 15. Abre el archivo en el editor de codigo.
- 16. Elimina el contenido inicial generado por Unity.
- 17. Escribe los using y la clase base como se muestra.

## 5.1 Using necesarios

```
using System;
using System.Collections;
using System.Globalization;
using TMPro;
using UnityEngine;
using UnityEngine.Networking;
using UnityEngine.UI;
```

| Namespace              | Uso                                                           |
|------------------------|---------------------------------------------------------------|
| System                 | Action, Array y tipos base.                                   |
| System.Collections     | IEnumerator para corrutinas.                                  |
| System.Globalization   | Numeros con CultureInfo.InvariantCulture en prompts tecnicos. |
| TMPro                  | TMP_InputField y TextMeshProUGUI.                             |
| UnityEngine            | MonoBehaviour, AudioClip, Mathf y Debug.                      |
| UnityEngine.Networking | UnityWebRequest y handlers HTTP.                              |
| UnityEngine.UI         | Button y eventos de UI.                                       |

## 5.2 Cabecera de la clase

```
public class GeminiTimbreGenerator : MonoBehaviour
{
    public const int WavetableSize = 2048;
    private const string GeminiEndpoint = "https://generativelanguage.googleapis.com/v1beta/interactions";
```

WavetableSize coincide con Osc. GeminiEndpoint apunta al endpoint REST de Interactions. La clave real se asigna en el Inspector, no en el codigo.

# 6 Estructuras de datos y configuracion

## 6.1 Campos visibles en Inspector

```
[Header("Target synth")]
public Osc targetOsc;

[Header("Gemini")]
[Tooltip("Assign each student's Gemini API key in the Inspector for local testing. Do not commit scenes or prefabs that contain real keys.")]
public string geminiApiKey = string.Empty;
public string modelName = "gemini-3.8-flash";
[Range(1, 120)]
public int requestTimeoutSeconds = 30;
public bool applyFallbackWhenGeminiFails = false;

[Header("Prompt UI")]
public TMP_InputField promptInput;
public Button generatePromptButton;

[TextArea(2, 5)]
public string fallbackPrompt = "Warm analog bass with fast attack, strong fundamental, few high harmonics and slow release.";
```

```
[Header("Audio UI")]
public AudioClip referenceAudioClip;
public Button analyzeAudioButton;
public bool useGeminiToComplementAudioAnalysis = false;

[Header("Status")]
public TextMeshProUGUI statusText;
public bool logDetails = true;
```

targetOsc es el sintetizador destino. modelName permite cambiar de modelo. requestTimeoutSeconds evita esperas indefinidas. Los campos de UI son opcionales pero permiten probar prompt y AudioClip desde botones.

6.2 Estructuras publicas

```
public class HarmonicDescriptor
{
    public int index = 1;
    public float amplitude = 1f;
    public float phaseRadians = 0f;
}

public class TimbreDescriptor
{
    public float brightness = 0.35f;
    public float harmonicity = 0.8f;
    public float noisiness = 0.05f;
    public float warmth = 0.5f;
    public float fundamentalStrength = 1f;
    public HarmonicDescriptor[] harmonics = Array.Empty<HarmonicDescriptor>();
}

public class ADSRDescriptor
{
    public float attackMs = 10f;
    public float decayMs = 180f;
    public float sustainMs = 600f;
    public float sustainLevel = 0.65f;
    public float releaseMs = 300f;
}

public class GeneratedTimbre
{
    public TimbreDescriptor timbre = new TimbreDescriptor();
    public ADSRDescriptor adsr = new ADSRDescriptor();
    public float[] wavetable = Array.Empty<float>();
    public string source = "Unknown";
    public bool usedGemini;
    public bool usedLocalDsp;
    public string message = string.Empty;

    public bool IsUsable()
    {
        return wavetable != null && wavetable.Length == WavetableSize && adsr != null;
    }
}
```

| Estructura         | Representa                                                      | Quien la escribe                 | Quien la consume              |
|--------------------|-----------------------------------------------------------------|----------------------------------|-------------------------------|
| HarmonicDescriptor | Un parcial: indice armonico, amplitud 0 a 1 y fase en radianes. | Gemini, fallback o DSP local.    | BuildWavetableFromDescriptor. |
| TimbreDescriptor   | Rasgos timbricos y arreglo de armonicos.                        | Gemini o estimacion local.       | El builder de wavetable.      |
| ADSRDescriptor     | Attack, decay, sustain, release en ms y sustainLevel 0 a 1.     | Gemini o analisis de envolvente. | Osc.ApplyGeneratedTimbre.     |
| GeneratedTimbre    | Paquete final wavetable + ADSR + metadata.                      | GeminiTimbreGenerator.           | Osc.                          |

## 6.3 DTO privados para respuestas de Gemini

```
[Serializable]
private class GeminiStructuredTimbre
{
    public TimbreDescriptor timbre = new TimbreDescriptor();
    public ADSRDescriptor adsr = new ADSRDescriptor();
    public HarmonicDescriptor[] harmonics = Array.Empty<HarmonicDescriptor>();
}

#pragma warning disable 0649
[Serializable]
private class GeminiGenerateContentResponse
{
    public GeminiCandidate[] candidates;
    public GeminiError error;
}

[Serializable]
private class GeminiInteractionResponse
{
    public GeminiInteraction interaction;
    public string outputText;
    public string output_text;
    public GeminiError error;
}

[Serializable]
private class GeminiInteraction
{
    public string outputText;
    public string output_text;
}

[Serializable]
private class GeminiCandidate
{
    public GeminiContent content;
    public string finishReason;
}

[Serializable]
private class GeminiContent
{
    public GeminiPart[] parts;
    public string role;
}

[Serializable]
private class GeminiPart
{
    public string text;
}

[Serializable]
private class GeminiError
{
    public int code;
    public string message;
    public string status;
}

#pragma warning restore 0649
```

Estos tipos modelan formas posibles de respuesta HTTP. No forman parte del motor de audio.

## 6.4 Ciclo de vida y botones

```
private Coroutine runningRequest;
private GeneratedTimbre lastValidTimbre;

private void Awake()
{
    if (targetOsc == null)
        targetOsc = FindAnyObjectByType<Osc>();
}

private void OnEnable()
{
    if (generatePromptButton != null)
        generatePromptButton.onClick.AddListener(GenerateFromPromptUI);
}
```

```

        if (analyzeAudioButton != null)
            analyzeAudioButton.onClick.AddListener(GenerateFromAudioUI);
    }

    private void OnDisable()
    {
        if (generatePromptButton != null)
            generatePromptButton.onClick.RemoveListener(GenerateFromPromptUI);

        if (analyzeAudioButton != null)
            analyzeAudioButton.onClick.RemoveListener(GenerateFromAudioUI);
    }

```

## 7 Conexion con Gemini y structured output

### 7.1 Leer la API key pegada en el código

```

string apiKey = string.IsNullOrEmpty(geminiApiKey) ? string.Empty : geminiApiKey.Trim();
if (string.IsNullOrEmpty(apiKey))
{
    ReportFailure("Configura geminiApiKey en el Inspector antes de llamar a Gemini.");

    if (applyFallbackWhenGeminiFails)
        ApplyGeneratedTimbre(BuildFallbackTimbre(prompt, "Fallback local: falta geminiApiKey."), onGenerated);

    yield break;
}

```

La primera parte evita un `NullReferenceException` si el campo está vacío. `Trim` elimina espacios accidentales al inicio o al final. Si el campo está vacío, el método reporta el fallo y no toca el sintetizador.

### 7.2 Métodos de entrada para prompt

```

[ContextMenu("Generate From Prompt")]
public void GenerateFromPromptUI()
{
    string prompt = promptInput != null ? promptInput.text : fallbackPrompt;
    GenerateFromPrompt(prompt);
}

[ContextMenu("Generate From AudioClip")]
public void GenerateFromAudioUI()
{
    GenerateFromAudio(referenceAudioClip);
}

public void GenerateFromPrompt(string prompt)
{
    if (runningRequest != null)
        StopCoroutine(runningRequest);

    runningRequest = StartCoroutine(GenerateFromPromptCoroutine(prompt));
}

```

### 7.3 Corrutina completa de prompt

```

public IEnumerator GenerateFromPromptCoroutine(string prompt, Action<GeneratedTimbre> onGenerated = null)
{
    if (string.IsNullOrEmpty(prompt))
    {
        ReportFailure("Prompt vacío. No se generó un timbre nuevo.");
        yield break;
    }

    string apiKey = string.IsNullOrEmpty(geminiApiKey) ? string.Empty : geminiApiKey.Trim();
    if (string.IsNullOrEmpty(apiKey))
    {
        ReportFailure("Configura geminiApiKey en el Inspector antes de llamar a Gemini.");
        Tutorial de implementacion de wavetables y ADSR asistidas por IA
    }
}

```



```

        if (applyFallbackWhenGeminifails)
            ApplyGeneratedTimbre(BuildFallbackTimbre(prompt, "Fallback local: falta geminiApiKey."), onGenerated);

        yield break;
    }

    SetStatus("Solicitando descriptor timbrico a Gemini...");

    using (UnityWebRequest request = BuildGeminirequest(prompt, apiKey))
    {
        yield return request.SendWebRequest();

        if (request.result != UnityWebRequest.Result.Success)
        {
            string error = BuildHttpErrorMessage(request);
            ReportFailure(error);

            if (applyFallbackWhenGeminifails)
                ApplyGeneratedTimbre(BuildFallbackTimbre(prompt, error), onGenerated);

            yield break;
        }

        if (!TryParseGeminireponse(request.downloadHandler.text, out TimbreDescriptor descriptor, out ADSRDescriptor adsr, out
string parseError))
        {
            ReportFailure(parseError);

            if (applyFallbackWhenGeminifails)
                ApplyGeneratedTimbre(BuildFallbackTimbre(prompt, parseError), onGenerated);

            yield break;
        }

        GeneratedTimbre generated = BuildGeneratedTimbre(descriptor, adsr, "Prompt", true, false, "Generado desde prompt con
Gemini.");
        ApplyGeneratedTimbre(generated, onGenerated);
    }
}

```

## 7.4 Crear UnityWebRequest

```

private UnityWebRequest BuildGeminirequest(string prompt, string apiKey)
{
    string body = BuildGeminirequestjson(prompt);
    byte[] bodyBytes = System.Text.Encoding.UTF8.GetBytes(body);

    UnityWebRequest request = new UnityWebRequest(GeminiEndpoint, UnityWebRequest.kHttpVerbPOST);
    request.uploadHandler = new UploadHandlerRaw(bodyBytes);
    request.downloadHandler = new DownloadHandlerBuffer();
    request.timeout = Mathf.Max(1, requestTimeoutSeconds);
    request.SetRequestHeader("Content-Type", "application/json");
    request.SetRequestHeader("x-goog-api-key", apiKey);
    return request;
}

```

## 7.5 Body y prompt instruccional

```

private string BuildGeminirequestjson(string prompt)
{
    string instructionalPrompt =
        "You are helping a Unity procedural synthesizer create a wavetable and ADSR envelope. " +
        "Return only structured JSON that matches the schema. " +
        "Do not generate audio samples. Do not return 2048 wavetable values. " +
        "Describe the sound using harmonic amplitudes, phases, timbre values, and ADSR values. " +
        "ADSR units must be milliseconds for attackMs, decayMs, sustainMs, releaseMs, and 0..1 for sustainLevel. " +
        "Keep harmonics musically useful and compact, preferably 1 to 24 harmonics. " +
        "User sound description: " + prompt;

    return "{
        + \"model\": \"\" + JsonEscape(modelName) + "\",
        + \"input\": \"\" + JsonEscape(instructionalPrompt) + "\",
        + \"response_format\": {
        + \"type\": \"text\",

```

```

+ "\"mime_type\": \"application/json\", \"
+ "\"schema\": \" + BuildResponseSchemaJson()
+ \"}\";
+ \"}\";
}

```

No se solicita float[2048]. Gemini devuelve datos compactos y el DSP local construye el arreglo grande.

## 7.6 Schema real de structured output

```

private static string BuildResponseSchemaJson()
{
    return "{
        + \"type\": \"object\",
        + \"properties\": {
        + \"timbre\": { \"type\": \"object\", \"properties\": {
        + \"brightness\": { \"type\": \"number\", \"minimum\": 0, \"maximum\": 1 },
        + \"harmonicity\": { \"type\": \"number\", \"minimum\": 0, \"maximum\": 1 },
        + \"noisiness\": { \"type\": \"number\", \"minimum\": 0, \"maximum\": 1 },
        + \"warmth\": { \"type\": \"number\", \"minimum\": 0, \"maximum\": 1 },
        + \"fundamentalStrength\": { \"type\": \"number\", \"minimum\": 0, \"maximum\": 1 }
        + }, \"required\": [ \"brightness\", \"harmonicity\", \"noisiness\", \"warmth\", \"fundamentalStrength\" ] },
        + \"harmonics\": { \"type\": \"array\", \"minItems\": 1, \"maxItems\": 32, \"items\": { \"type\": \"object\", \"properties\": {
        + \"index\": { \"type\": \"integer\", \"minimum\": 1, \"maximum\": 64 },
        + \"amplitude\": { \"type\": \"number\", \"minimum\": 0, \"maximum\": 1 },
        + \"phaseRadians\": { \"type\": \"number\", \"minimum\": -6.283185, \"maximum\": 6.283185 }
        + }, \"required\": [ \"index\", \"amplitude\", \"phaseRadians\" ] } },
        + \"adsr\": { \"type\": \"object\", \"properties\": {
        + \"attackMs\": { \"type\": \"number\", \"minimum\": 1, \"maximum\": 400 },
        + \"decayMs\": { \"type\": \"number\", \"minimum\": 1, \"maximum\": 1000 },
        + \"sustainMs\": { \"type\": \"number\", \"minimum\": 0, \"maximum\": 5000 },
        + \"sustainLevel\": { \"type\": \"number\", \"minimum\": 0.001, \"maximum\": 1 },
        + \"releaseMs\": { \"type\": \"number\", \"minimum\": 1, \"maximum\": 1000 }
        + }, \"required\": [ \"attackMs\", \"decayMs\", \"sustainMs\", \"sustainLevel\", \"releaseMs\" ] }
        + },
        + \"required\": [ \"timbre\", \"harmonics\", \"adsr\" ]
    }";
}

```

## 7.7 Ejemplo de respuesta JSON compatible

```

{
  "timbre": {
    "brightness": 0.32,
    "harmonicity": 0.88,
    "noisiness": 0.03,
    "warmth": 0.82,
    "fundamentalStrength": 1.0
  },
  "harmonics": [
    { "index": 1, "amplitude": 1.0, "phaseRadians": 0.0 },
    { "index": 2, "amplitude": 0.28, "phaseRadians": 0.0 },
    { "index": 3, "amplitude": 0.18, "phaseRadians": 0.0 },
    { "index": 5, "amplitude": 0.08, "phaseRadians": 0.0 }
  ],
  "adsr": {
    "attackMs": 8,
    "decayMs": 220,
    "sustainMs": 900,
    "sustainLevel": 0.68,
    "releaseMs": 820
  }
}

```

## 7.8 Errores HTTP

```

private static string BuildHttpErrorMessage(UnityWebRequest request)
{
    long code = request.responseCode;
    string prefix;

    if (code == 401 || code == 403)
        prefix = "Gemini rechazó la credencial o los permisos.";
    else if (code == 429)
        prefix = "Gemini reportó límite de cuota o rate limit.";
    else if (code >= 500)
        prefix = "Gemini no está disponible temporalmente.";
    else if (request.result == UnityWebRequest.Result.ConnectionError)
        prefix = "No hay conexión con Gemini.";
    else if (request.result == UnityWebRequest.Result.ProtocolError)

```

```

        prefix = "Gemini devolvió un error HTTP.";
    else if (request.result == UnityWebRequest.Result.DataProcessingError)
        prefix = "No se pudo procesar la respuesta de Gemini.";
    else
        prefix = "La solicitud a Gemini falló.";

    return prefix + " HTTP " + code + ". " + request.error;
}

```

| Situacion                | Sintoma                            | Correccion                                         |
|--------------------------|------------------------------------|----------------------------------------------------|
| No existe GEMINI_API_KEY | Mensaje de variable no encontrada. | Configura setx y reinicia Unity.                   |
| Key invalida o permisos  | HTTP 401 o 403.                    | Crea una key nueva o revisa permisos del proyecto. |
| Rate limit               | HTTP 429.                          | Espera o reduce pruebas.                           |
| Sin Internet             | ConnectionError.                   | Revisa red, proxy o firewall.                      |
| Modelo no disponible     | Error de modelo.                   | Cambia modelName por un modelo disponible.         |

## Checkpoint 2 Unity puede comunicarse con Gemini

### Que debe funcionar

La corrutina llega a SendWebRequest y recibe una respuesta HTTP.

### Como probarlo

Ejecuta Generate From Prompt con una key configurada.

### Resultado esperado

El status debe cambiar a Solicitando descriptor timbrico a Gemini.

### Errores frecuentes

- HTTP 401 o 403 indica credencial o permisos.
- ConnectionError indica red.
- Si no cambia el status, el componente no esta activo.

## 8 Parsing validacion y fallback

```

private bool TryParseGeminiResponse(string responseJson, out TimbreDescriptor descriptor, out ADSRDescriptor adsr, out string error)
{
    descriptor = null;
    adsr = null;
    error = string.Empty;

    if (string.IsNullOrEmpty(responseJson))
    {
        error = "Gemini devolvió una respuesta vacía.";
        return false;
    }

    GeminiInteractionResponse interactionResponse;
    try
    {
        interactionResponse = JsonUtility.FromJson<GeminiInteractionResponse>(responseJson);
    }
    catch (Exception exception)
    {
        error = "No se pudo interpretar la respuesta HTTP de Gemini: " + exception.Message;
        return false;
    }

    if (interactionResponse != null && interactionResponse.error != null &&
        !string.IsNullOrEmpty(interactionResponse.error.message))
    {
        error = "Gemini reportó error " + interactionResponse.error.code + ": " + interactionResponse.error.message;
        return false;
    }

    string structuredJson = ExtractInteractionText(interactionResponse);

    if (string.IsNullOrEmpty(structuredJson))

```

```

        structuredJson = ExtractGenerateContentText(responseJson, out error);

    if (string.IsNullOrEmpty(structuredJson) && LooksLikeStructuredTimbreJson(responseJson))
        structuredJson = responseJson;

    if (string.IsNullOrEmpty(structuredJson))
    {
        if (string.IsNullOrEmpty(error))
            error = "Gemini no devolvió texto estructurado utilizable.";

        return false;
    }

    return TryParseStructuredTimbreJson(structuredJson, out descriptor, out adsr, out error);
}

private bool TryParseStructuredTimbreJson(string structuredJson, out TimbreDescriptor descriptor, out ADSRDescriptor adsr, out string error)
{
    descriptor = null;
    adsr = null;
    error = string.Empty;

    if (!LooksLikeStructuredTimbreJson(structuredJson))
    {
        error = "Gemini no devolvió el contrato JSON esperado para timbre, armónicos y ADSR.";
        return false;
    }

    GeminiStructuredTimbre structured;
    try
    {
        structured = JsonUtility.FromJson<GeminiStructuredTimbre>(structuredJson);
    }
    catch (Exception exception)
    {
        error = "Gemini devolvió JSON inválido: " + exception.Message;
        return false;
    }

    if (structured == null)
    {
        error = "Gemini devolvió JSON vacío.";
        return false;
    }

    descriptor = structured.timbre ?? new TimbreDescriptor();
    if (structured.harmonics == null || structured.harmonics.Length == 0)
    {
        error = "Gemini devolvió un descriptor sin armónicos.";
        return false;
    }

    descriptor.harmonics = structured.harmonics;
    adsr = structured.adsr ?? new ADSRDescriptor();

    ValidateDescriptor(descriptor);
    ValidateAdsr(adsr);
    return true;
}

private static bool LooksLikeStructuredTimbreJson(string json)
{
    if (string.IsNullOrEmpty(json))
        return false;

    return json.Contains("\"timbre\"") &&
        json.Contains("\"harmonics\"") &&
        json.Contains("\"adsr\"");
}

private static string ExtractInteractionText(GeminiInteractionResponse response)
{
    if (response == null)
        return string.Empty;

    if (!string.IsNullOrEmpty(response.outputText))
        return response.outputText;
}

```

```

        if (!string.IsNullOrEmpty(response.output_text))
            return response.output_text;

        if (response.interaction == null)
            return string.Empty;

        if (!string.IsNullOrEmpty(response.interaction.outputText))
            return response.interaction.outputText;

        return response.interaction.output_text ?? string.Empty;
    }

    private static string ExtractGenerateContentText(string responseJson, out string error)
    {
        error = string.Empty;

        GeminiGenerateContentResponse response;
        try
        {
            response = JsonUtility.FromJson<GeminiGenerateContentResponse>(responseJson);
        }
        catch (Exception exception)
        {
            error = "No se pudo interpretar una respuesta legacy de Gemini: " + exception.Message;
            return string.Empty;
        }

        if (response != null && response.error != null && !string.IsNullOrEmpty(response.error.message))
        {
            error = "Gemini reportó error " + response.error.code + ": " + response.error.message;
            return string.Empty;
        }

        if (response == null || response.candidates == null || response.candidates.Length == 0)
            return string.Empty;

        GeminiContent content = response.candidates[0].content;
        if (content == null || content.parts == null || content.parts.Length == 0)
            return string.Empty;

        return content.parts[0].text ?? string.Empty;
    }

    private static void ValidateDescriptor(TimbreDescriptor descriptor)
    {
        if (descriptor == null)
            return;

        descriptor.brightness = Mathf.Clamp01(descriptor.brightness);
        descriptor.harmonicity = Mathf.Clamp01(descriptor.harmonicity);
        descriptor.noisiness = Mathf.Clamp01(descriptor.noisiness);
        descriptor.warmth = Mathf.Clamp01(descriptor.warmth);
        descriptor.fundamentalStrength = Mathf.Clamp01(descriptor.fundamentalStrength);

        if (descriptor.harmonics == null || descriptor.harmonics.Length == 0)
            descriptor.harmonics = BuildDefaultHarmonics(descriptor.brightness, descriptor.warmth, descriptor.fundamentalStrength);

        int count = Mathf.Clamp(descriptor.harmonics.Length, 1, 64);
        if (count != descriptor.harmonics.Length)
            Array.Resize(ref descriptor.harmonics, count);

        for (int i = 0; i < descriptor.harmonics.Length; i++)
        {
            if (descriptor.harmonics[i] == null)
                descriptor.harmonics[i] = new HarmonicDescriptor { index = i + 1, amplitude = 0f, phaseRadians = 0f };

            descriptor.harmonics[i].index = Mathf.Clamp(descriptor.harmonics[i].index, 1, 128);
            descriptor.harmonics[i].amplitude = Mathf.Clamp01(descriptor.harmonics[i].amplitude);
            descriptor.harmonics[i].phaseRadians = Mathf.Clamp(descriptor.harmonics[i].phaseRadians, -Mathf.PI * 2f, Mathf.PI * 2f);
        }
    }

    private static void ValidateAdsr(ADSRDescriptor adsr)
    {
        if (adsr == null)
            return;
    }

```

```

        adsr.attackMs = Mathf.Clamp(adsr.attackMs, 1f, 400f);
        adsr.decayMs = Mathf.Clamp(adsr.decayMs, 1f, 1000f);
        adsr.sustainMs = Mathf.Clamp(adsr.sustainMs, 0f, 5000f);
        adsr.sustainLevel = Mathf.Clamp(adsr.sustainLevel, 0.001f, 1f);
        adsr.releaseMs = Mathf.Clamp(adsr.releaseMs, 1f, 1000f);
    }

```

El parser acepta outputText de Interactions, una respuesta legacy con candidates o JSON directo. La validacion local aplica clamps, limita cantidad de armonicos y rechaza respuestas sin contrato utilizable.

```

private GeneratedTimbre BuildFallbackTimbre(string prompt, string message)
{
    string lower = prompt.ToLowerInvariant();
    float brightness = lower.Contains("bright") || lower.Contains("metallic") ? 0.8f : 0.35f;
    float warmth = lower.Contains("warm") || lower.Contains("analog") ? 0.8f : 0.45f;
    float noisiness = lower.Contains("noise") || lower.Contains("distorted") ? 0.25f : 0.04f;
    float fundamental = lower.Contains("bass") || lower.Contains("fundamental") ? 1f : 0.75f;

    TimbreDescriptor descriptor = new TimbreDescriptor
    {
        brightness = brightness,
        harmonicity = 0.85f,
        noisiness = noisiness,
        warmth = warmth,
        fundamentalStrength = fundamental,
        harmonics = BuildDefaultHarmonics(brightness, warmth, fundamental)
    };

    ADSRDescriptor adsr = new ADSRDescriptor
    {
        attackMs = lower.Contains("slow attack") || lower.Contains("pad") ? 220f : 10f,
        decayMs = 180f,
        sustainMs = lower.Contains("pad") ? 1600f : 650f,
        sustainLevel = lower.Contains("pluck") ? 0.3f : 0.68f,
        releaseMs = lower.Contains("slow release") || lower.Contains("pad") ? 850f : 240f
    };

    return BuildGeneratedTimbre(descriptor, adsr, "PromptFallback", false, true, message);
}

private static HarmonicDescriptor[] BuildDefaultHarmonics(float brightness, float warmth, float fundamentalStrength)
{
    int count = Mathf.RoundToInt(Mathf.Lerp(6f, 18f, Mathf.Clamp01(brightness)));
    HarmonicDescriptor[] harmonics = new HarmonicDescriptor[count];

    for (int i = 0; i < count; i++)
    {
        int harmonicIndex = i + 1;
        float slope = Mathf.Lerp(1.8f, 0.75f, Mathf.Clamp01(brightness));
        float amplitude = Mathf.Pow(1f / harmonicIndex, slope);

        if (harmonicIndex == 1)
            amplitude *= Mathf.Lerp(0.7f, 1.25f, Mathf.Clamp01(fundamentalStrength));

        if (warmth > 0.55f && harmonicIndex % 2 == 0)
            amplitude *= 0.65f;

        harmonics[i] = new HarmonicDescriptor
        {
            index = harmonicIndex,
            amplitude = Mathf.Clamp01(amplitude),
            phaseRadians = 0f
        };
    }

    return harmonics;
}

```

## Checkpoint 3 Gemini devuelve JSON estructurado

### Que debe funcionar

TryParseGeminiResponse obtiene TimbreDescriptor y ADSRDescriptor.

### Como probarlo

Ejecuta prompt y revisa que no aparezca contrato JSON invalido.

## Resultado esperado

El sistema debe avanzar a BuildGeneratedTimbre.

## Errores frecuentes

- El modelo responde lenguaje natural: revisa response\_format.
- Los nombres JSON no coinciden: JsonUtility no llenara campos.
- harmonics vacio: se rechaza por seguridad.

# 9 Construcción local de wavetable

La wavetable se genera mediante síntesis aditiva:

$$x[n] = \sum A_k * \sin(2\pi * k * n / N + \phi_k)$$

| Variable       | Significado                               |
|----------------|-------------------------------------------|
| n              | Indice de muestra dentro de la wavetable. |
| N              | Tamaño de la wavetable. Aquí N es 2048.   |
| k              | Indice armónico. 1 es la fundamental.     |
| A <sub>k</sub> | Amplitud del armónico k.                  |
| φ <sub>k</sub> | Fase del armónico en radianes.            |

```
public static float[] BuildWavetableFromDescriptor(TimbreDescriptor descriptor)
{
    descriptor = descriptor ?? new TimbreDescriptor();
    ValidateDescriptor(descriptor);

    float[] wavetable = new float[WavetableSize];
    HarmonicDescriptor[] harmonics = descriptor.harmonics;

    for (int n = 0; n < WavetableSize; n++)
    {
        float phase01 = (float)n / WavetableSize;
        float value = 0f;

        for (int i = 0; i < harmonics.Length; i++)
        {
            HarmonicDescriptor harmonic = harmonics[i];
            int index = Mathf.Max(1, harmonic.index);
            float amplitude = Mathf.Clamp01(harmonic.amplitude);
            float phase = harmonic.phaseRadians;

            value += amplitude * Mathf.Sin((2f * Mathf.PI * index * phase01) + phase);
        }

        wavetable[n] = value;
    }

    NormalizeInPlace(wavetable);
    return wavetable;
}

private static void NormalizeInPlace(float[] data, bool positiveOnly = false)
{
    if (data == null || data.Length == 0)
        return;

    float max = 0f;
    for (int i = 0; i < data.Length; i++)
        max = Mathf.Max(max, positiveOnly ? data[i] : Mathf.Abs(data[i]));

    if (max <= 0.000001f)
        return;

    for (int i = 0; i < data.Length; i++)
        data[i] = Mathf.Clamp(data[i] / max, positiveOnly ? 0f : -1f, 1f);
}
```

```
}
```

## Checkpoint 4 El JSON genera una wavetable

### Que debe funcionar

BuildGeneratedTimbre crea float[2048] normalizado.

### Como probarlo

Ejecuta Generate Local Test Timbre para probar sin red.

### Resultado esperado

La wavetable debe tener 2048 muestras entre -1 y 1.

### Errores frecuentes

- Todos los armonicos con amplitud cero producen silencio.
- Valores fuera de rango se corrigen con Clamp.
- Un arreglo vacio de Gemini se rechaza antes.

## 10 ADSR y GeneratedTimbre

```
public GeneratedTimbre BuildGeneratedTimbre(TimbreDescriptor descriptor, ADSRDescriptor adsr, string source, bool usedGemini,
bool usedLocalDsp, string message)
{
    descriptor = descriptor ?? new TimbreDescriptor();
    adsr = adsr ?? new ADSRDescriptor();

    ValidateDescriptor(descriptor);
    ValidateAdsr(adsr);

    return new GeneratedTimbre
    {
        timbre = descriptor,
        adsr = adsr,
        wavetable = BuildWavetableFromDescriptor(descriptor),
        source = source,
        usedGemini = usedGemini,
        usedLocalDsp = usedLocalDsp,
        message = message
    };
}

private void ApplyGeneratedTimbre(GeneratedTimbre generated, Action<GeneratedTimbre> onGenerated)
{
    if (generated == null || !generated.IsUsable())
    {
        ReportFailure("El timbre generado no es utilizable.");
        return;
    }

    if (targetOsc == null)
        targetOsc = FindAnyObjectByType<Osc>();

    if (targetOsc != null)
        targetOsc.ApplyGeneratedTimbre(generated);

    lastValidTimbre = generated;
    onGenerated?.Invoke(generated);
    SetStatus(generated.message);

    if (logDetails)
        Debug.Log("[GeminiTimbreGenerator] " + generated.message);
}
```

ADSRDescriptor usa milisegundos para attackMs, decayMs, sustainMs y releaseMs. sustainLevel es 0 a 1.  
GeneratedTimbre junta wavetable y ADSR para enviarlos a Osc.



## 11 Modificar Osc.cs

Agrega este metodo dentro de la clase Osc, cerca de ToggleWavetable y antes de los campos ADSR. No borres los loaders manuales ni los controles existentes.

```
public void ApplyGeneratedTimbre(GeminiTimbreGenerator.GeneratedTimbre generatedTimbre)
{
    if (generatedTimbre == null || generatedTimbre.wavetable == null || generatedTimbre.wavetable.Length == 0 ||
    generatedTimbre.adsr == null)
    {
        Debug.LogWarning("[Osc] Generated timbre is not valid. Current synth settings were preserved.");
        return;
    }

    float[] generatedWavetable = new float[wavetableSize];
    int sourceLength = generatedTimbre.wavetable.Length;

    for (int i = 0; i < wavetableSize; i++)
    {
        float readPos = ((float)i / wavetableSize) * sourceLength;
        int index0 = Mathf.FloorToInt(readPos) % sourceLength;
        int index1 = (index0 + 1) % sourceLength;
        float frac = readPos - Mathf.Floor(readPos);
        generatedWavetable[i] = Mathf.Clamp(Mathf.Lerp(generatedTimbre.wavetable[index0],
        generatedTimbre.wavetable[index1], frac), -1f, 1f);
    }

    wavetable = generatedWavetable;
    FormType = WaveFormType.Custom1;
    useWavetable = true;
    currentWavetableLoadedFromExternal = true;
    useAudioClipADSR = false;

    A = Mathf.Clamp(generatedTimbre.adsr.attackMs, 1f, 400f);
    D = Mathf.Clamp(generatedTimbre.adsr.decayMs, 1f, 1000f);
    S = Mathf.Clamp(generatedTimbre.adsr.sustainMs, 0f, 5000f);
    SL = Mathf.Clamp(generatedTimbre.adsr.sustainLevel, 0.001f, 1f);
    R = Mathf.Clamp(generatedTimbre.adsr.releaseMs, 1f, 1000f);
    SLegacy = SL;

    UpdateADSR();
    MarkExternalBackendDirty();
}
```

El metodo usa variables existentes: wavetable, wavetableSize, FormType, useWavetable, currentWavetableLoadedFromExternal, useAudioClipADSR, A, D, S, SL, R, SLegacy, UpdateADSR y MarkExternalBackendDirty.

```
GeminiTimbreGenerator
|
v
GeneratedTimbre
|
v
Osc.ApplyGeneratedTimbre
|
v
wavetable interna + A D S SL R
|
v
ProceduralSynthVoiceConfig y render existente
```

### Checkpoint 5 Osc reproduce la wavetable generada

#### Que debe funcionar

Osc recibe GeneratedTimbre y actualiza wavetable y ADSR.

#### Como probarlo

Guarda scripts, vuelve a Unity y ejecuta Generate Local Test Timbre.

## Resultado esperado

Una nota debe sonar usando Custom1 y ADSR generado.

## Errores frecuentes

- Error de tipo GeminiTimbreGenerator: el script nuevo no compilo.
- No suena: targetOsc no esta asignado.
- Suena igual: FormType fue cambiado despues.

## 12 Primera prueba reproducible con prompt

18. Guarda GeminiTimbreGenerator.cs y Osc.cs.
19. Vuelve a Unity y espera compilacion.
20. Crea un GameObject vacio para el generador.
21. Agrega el componente GeminiTimbreGenerator.
22. Arrastra el objeto con Osc al campo targetOsc.
23. Confirma que geminiApiKey contiene tu API key real en el Inspector.
24. Escribe el prompt Warm dark analog bass with fast attack and slow release.
25. Presiona Generate o ejecuta Generate From Prompt.
26. Reproduce una nota.

Resultado esperado: el status indica que se genero desde prompt con Gemini. El sonido cambia hacia un bajo calido con ataque rapido y release lento.

## 13 Analisis DSP de AudioClip

El analisis de audio es local. No se envia el archivo a Gemini. El pipeline lee muestras, convierte a mono, normaliza, calcula envolvente, estima ADSR, detecta periodo, extrae ciclo y remuestrea a 2048 muestras.

### Entrada AudioClip a GeneratedTimbre

```
private bool TryAnalyzeAudioClip(AudioClip clip, out GeneratedTimbre generated, out string error)
{
    generated = null;
    error = string.Empty;

    if (clip.samples <= 0 || clip.channels <= 0)
    {
        error = "El AudioClip no contiene muestras válidas.";
        return false;
    }

    float[] raw = new float[clip.samples * clip.channels];
    if (!clip.GetData(raw, 0))
    {
        error = "No se pudo leer el AudioClip. Revisa que el clip permita cargar sus datos.";
        return false;
    }

    float[] mono = ConvertToMonoAndNormalize(raw, clip.channels);
    if (mono.Length == 0)
    {
        error = "No se pudieron convertir muestras del AudioClip a mono.";
        return false;
    }

    float[] envelope = BuildSmoothedEnvelope(mono, clip.frequency);
    ADSRDescriptor adsr = EstimateAdsrFromEnvelope(envelope, clip.frequency);

    int period = EstimatePeriodByAutocorrelation(mono, envelope, clip.frequency);
    float[] wavetable = period > 0
        ? ExtractCycleAsWavetable(mono, envelope, period)
        : ResampleWindowAsWavetable(mono, FindStableStart(envelope));

    TimbreDescriptor descriptor = EstimateDescriptorFromAudio(wavetable, adsr);

    generated = new GeneratedTimbre
```

```

{
    timbre = descriptor,
    adsr = adsr,
    wavetable = wavetable,
    source = "AudioClip",
    usedGemini = false,
    usedLocalDsp = true,
    message = "Generado desde AudioClip con DSP local."
};

return generated.IsUsable();
}

```

GetData copia muestras en memoria fuera del audio callback.

## Convertir a mono y normalizar

```

private static float[] ConvertToMonoAndNormalize(float[] raw, int channels)
{
    int safeChannels = Mathf.Max(1, channels);
    int frames = raw.Length / safeChannels;
    float[] mono = new float[frames];
    float maxAbs = 0f;

    for (int frame = 0; frame < frames; frame++)
    {
        float sum = 0f;
        int baseIndex = frame * safeChannels;

        for (int channel = 0; channel < safeChannels; channel++)
            sum += raw[baseIndex + channel];

        float value = sum / safeChannels;
        mono[frame] = value;
        maxAbs = Mathf.Max(maxAbs, Mathf.Abs(value));
    }

    if (maxAbs > 0.000001f)
    {
        for (int i = 0; i < mono.Length; i++)
            mono[i] /= maxAbs;
    }

    return mono;
}

```

Promedia canales y escala por amplitud maxima.

## Obtener envolvente suavizada

```

private static float[] BuildSmoothedEnvelope(float[] mono, int sampleRate)
{
    float[] envelope = new float[mono.Length];
    int window = Mathf.Clamp(sampleRate / 200, 16, 512);
    float sum = 0f;

    for (int i = 0; i < mono.Length; i++)
    {
        sum += Mathf.Abs(mono[i]);

        if (i >= window)
            sum -= Mathf.Abs(mono[i - window]);

        int count = Mathf.Min(i + 1, window);
        envelope[i] = sum / Mathf.Max(1, count);
    }

    NormalizeInPlace(envelope, true);
    return envelope;
}

```

Usa valor absoluto y promedio movil.

## Estimar ADSR

```

private static ADSRDescriptor EstimateAdsrFromEnvelope(float[] envelope, int sampleRate)
{

```

```

int peakIndex = 0;
float peak = 0f;

for (int i = 0; i < envelope.Length; i++)
{
    if (envelope[i] > peak)
    {
        peak = envelope[i];
        peakIndex = i;
    }
}

if (peak <= 0.0001f)
    return new ADSRDescriptor();

int onset = FindFirstAbove(envelope, peak * 0.08f, 0, envelope.Length - 1);
int attackEnd = FindFirstAbove(envelope, peak * 0.9f, onset, Mathf.Max(onset, peakIndex));
int end = FindLastAbove(envelope, peak * 0.08f);

float sustainLevel = EstimateSustainLevel(envelope, peakIndex, end);
int decayEnd = FindFirstBelow(envelope, sustainLevel + ((peak - sustainLevel) * 0.15f), peakIndex, end);
int releaseStart = FindReleaseStart(envelope, sustainLevel, peakIndex, end);

float attackMs = FramesToMs(Mathf.Max(1, attackEnd - onset), sampleRate);
float decayMs = FramesToMs(Mathf.Max(1, decayEnd - attackEnd), sampleRate);
float sustainMs = FramesToMs(Mathf.Max(0, releaseStart - decayEnd), sampleRate);
float releaseMs = FramesToMs(Mathf.Max(1, end - releaseStart), sampleRate);

ADSRDescriptor adsr = new ADSRDescriptor
{
    attackMs = attackMs,
    decayMs = decayMs,
    sustainMs = sustainMs,
    sustainLevel = sustainLevel,
    releaseMs = releaseMs
};

ValidateAdsr(adsr);
return adsr;
}

```

Detecta onset, ataque, decay, sustain y release con umbrales sobre la envolvente.

## Detectar pitch o periodo

```

private static int EstimatePeriodByAutocorrelation(float[] mono, float[] envelope, int sampleRate)
{
    int start = FindStableStart(envelope);
    int windowSize = Mathf.Min(4096, mono.Length - start);

    if (windowSize < 512)
        return -1;

    int minLag = Mathf.Max(1, sampleRate / 1200);
    int maxLag = Mathf.Min(sampleRate / 50, windowSize / 2);
    float bestScore = 0f;
    int bestLag = -1;

    for (int lag = minLag; lag <= maxLag; lag++)
    {
        float sum = 0f;
        float aEnergy = 0f;
        float bEnergy = 0f;
        int count = windowSize - lag;

        for (int i = 0; i < count; i++)
        {
            float a = mono[start + i];
            float b = mono[start + i + lag];
            sum += a * b;
            aEnergy += a * a;
            bEnergy += b * b;
        }

        float denom = Mathf.Sqrt(aEnergy * bEnergy);
        if (denom <= 0.000001f)
            continue;

        float score = sum / denom;
        if (score > bestScore)
        {

```

```

        bestScore = score;
        bestLag = lag;
    }
}

return bestScore >= 0.35f ? bestLag : -1;
}

```

Busca el lag que maximiza similitud normalizada.

## Extraer ciclo estable

```

private static float[] ExtractCycleAsWavetable(float[] mono, float[] envelope, int period)
{
    int stableStart = FindStableStart(envelope);
    int start = FindNearestRisingZeroCrossing(mono, stableStart, Mathf.Min(mono.Length - 1, stableStart + period));
    int end = Mathf.Min(mono.Length - 1, start + Mathf.Max(1, period));

    if (end <= start + 1)
        return ResampleWindowAsWavetable(mono, stableStart);

    float[] wavetable = new float[WavetableSize];
    int length = end - start;

    for (int i = 0; i < WavetableSize; i++)
    {
        float readPosition = start + ((float)i / WavetableSize) * length;
        int i0 = Mathf.Clamp(Mathf.FloorToInt(readPosition), 0, mono.Length - 1);
        int i1 = Mathf.Clamp(i0 + 1, 0, mono.Length - 1);
        float frac = readPosition - i0;
        wavetable[i] = Mathf.Lerp(mono[i0], mono[i1], frac);
    }

    RemoveDcAndNormalize(wavetable);
    return wavetable;
}

```

Busca cruce ascendente y remuestrea un periodo.

## Fallback de remuestreo

```

private static float[] ResampleWindowAsWavetable(float[] mono, int start)
{
    float[] wavetable = new float[WavetableSize];
    int safeStart = Mathf.Clamp(start, 0, Mathf.Max(0, mono.Length - 2));
    int available = Mathf.Max(1, mono.Length - safeStart - 1);
    int length = Mathf.Min(available, WavetableSize);

    for (int i = 0; i < WavetableSize; i++)
    {
        float readPosition = safeStart + ((float)i / WavetableSize) * length;
        int i0 = Mathf.Clamp(Mathf.FloorToInt(readPosition), 0, mono.Length - 1);
        int i1 = Mathf.Clamp(i0 + 1, 0, mono.Length - 1);
        float frac = readPosition - i0;
        wavetable[i] = Mathf.Lerp(mono[i0], mono[i1], frac);
    }

    RemoveDcAndNormalize(wavetable);
    return wavetable;
}

```

Usa una ventana estable si no hay periodo confiable.

## Quitar DC y normalizar

```

private static void RemoveDcAndNormalize(float[] data)
{
    if (data == null || data.Length == 0)
        return;

    float mean = 0f;
    for (int i = 0; i < data.Length; i++)
        mean += data[i];

    mean /= data.Length;

    for (int i = 0; i < data.Length; i++)

```

```

        data[i] -= mean;

    NormalizeInPlace(data);
}

```

Centra la onda y limita a -1..1.

## Descriptor aproximado desde audio

```

private static TimbreDescriptor EstimateDescriptorFromAudio(float[] wavetable, ADSRDescriptor adsr)
{
    float roughness = 0f;
    for (int i = 1; i < wavetable.Length; i++)
        roughness += Mathf.Abs(wavetable[i] - wavetable[i - 1]);

    roughness /= Mathf.Max(1, wavetable.Length - 1);

    float brightness = Mathf.Clamp01(roughness * 2.2f);
    float warmth = Mathf.Clamp01(1f - (brightness * 0.7f));
    float attackFastness = 1f - Mathf.Clamp01(adsr.attackMs / 400f);

    return new TimbreDescriptor
    {
        brightness = brightness,
        harmonicity = 0.7f,
        noisiness = Mathf.Clamp01(roughness * 0.6f),
        warmth = warmth,
        fundamentalStrength = Mathf.Lerp(0.7f, 1f, attackFastness),
        harmonics = BuildDefaultHarmonics(brightness, warmth, 1f)
    };
}

```

Calcula roughness, brightness y calidez aproximada.

## Checkpoint 6 AudioClip produce una wavetable

### Que debe funcionar

TryAnalyzeAudioClip devuelve GeneratedTimbre usable desde muestras locales.

### Como probarlo

Asigna un AudioClip tonal y ejecuta Generate From AudioClip.

### Resultado esperado

El status dice Generado desde AudioClip con DSP local y el sintetizador cambia de timbre.

### Errores frecuentes

- Clip no leible: revisa import settings.
- Clip silencioso: ADSR cae a valores por defecto.
- No hay pitch estable: se usa remuestreo de ventana.

## 14 Gemini como complemento del audio

Si useGeminiToComplementAudioAnalysis esta activo, la implementacion envia solo un resumen numerico local. No envia el archivo de audio completo.

```

private IEnumerator ComplementAudioAnalysisWithGemini(GeneratedTimbre generated)
{
    if (generated == null)
        yield break;

    string prompt = "Analyze this locally extracted audio descriptor and return a compact timbre descriptor. "
        + "Do not ask for audio samples. Local DSP already produced the wavetable. "
        + "Use the ADSR and rough timbre values to refine harmonics only. "
        + "Local values: brightness=" + generated.timbre.brightness.ToString("0.###", CultureInfo.InvariantCulture)
        + ", noisiness=" + generated.timbre.noisiness.ToString("0.###", CultureInfo.InvariantCulture)
        + ", attackMs=" + generated.adsr.attackMs.ToString("0.###", CultureInfo.InvariantCulture)
        + ", releaseMs=" + generated.adsr.releaseMs.ToString("0.###", CultureInfo.InvariantCulture) + ".";
}

```

```

        yield return GenerateFromPromptCoroutine(prompt);
    }

```

La llamada reutiliza GenerateFromPromptCoroutine. Si se quiere conservar exactamente la wavetable extraida del AudioClip, deja esta opcion apagada.

## 15 Segunda prueba reproducible con AudioClip

27. Importa un archivo WAV o MP3 corto.
28. Selecciona el AudioClip y verifica que Unity pueda cargar sus datos.
29. Asigna el clip a referenceAudioClip.
30. Deja useGeminiToComplementAudioAnalysis apagado para la primera prueba.
31. Ejecuta Generate From AudioClip o presiona Analyze Audio.
32. Reproduce una nota del sintetizador.

Resultado esperado: si el clip tiene tono estable, la wavetable extraida recuerda el color del audio. Si es percusivo o ruidoso, el resultado sera mas irregular.

## 16 Interfaz minima en Unity

### 16.1 Prompt

33. Crea un Canvas si no existe.
34. Crea UI > TextMeshPro > Input Field y llamalo PromptInput.
35. Crea UI > Button y llamalo GeneratePromptButton.
36. Crea UI > TextMeshPro Text y llamalo StatusText.
37. Arrastra PromptInput, GeneratePromptButton y StatusText a los campos del componente.
38. El script conecta OnClick en OnEnable. Si prefieres manual, en OnClick selecciona GeminiTimbreGenerator.GenerateFromPromptUI.

### 16.2 AudioClip

39. Crea un Button llamado AnalyzeAudioButton.
40. Arrastralo al campo analyzeAudioButton.
41. Arrastra un AudioClip importado al campo referenceAudioClip.
42. Si conectas OnClick manual, selecciona GeminiTimbreGenerator.GenerateFromAudioUI.

## 17 Audio thread

OnAudioFilterRead debe llenar buffers con latencia baja. Llamadas HTTP, parsing JSON, lectura de archivos, GetData pesado o autocorrelacion dentro del callback producen glitches.

Incorrecto:  
 OnAudioFilterRead -> UnityWebRequest -> JSON -> wavetable

Correcto:  
 UI/control thread -> Gemini o DSP local -> GeneratedTimbre validado -> Osc.ApplyGeneratedTimbre -> audio thread lee datos listos

La implementacion respeta esta separacion: UnityWebRequest y analisis DSP corren fuera del callback; Osc recibe una wavetable ya construida.

# 18 Codigo completo de referencia

## 18.1 Codigo completo de GeminiTimbreGenerator.cs

Compara tu archivo con esta version final.

```
using System;
using System.Collections;
using System.Globalization;
using TMPro;
using UnityEngine;
using UnityEngine.Networking;
using UnityEngine.UI;

public class GeminiTimbreGenerator : MonoBehaviour
{
    public const int WavetableSize = 2048;
    private const string GeminiEndpoint = "https://generativelanguage.googleapis.com/v1beta/interactions";

    [Header("Target synth")]
    public Osc targetOsc;

    [Header("Gemini")]
    [Tooltip("Assign each student's Gemini API key in the Inspector for local testing. Do not commit scenes or prefabs that contain real keys.")]
    public string geminiApiKey = string.Empty;
    public string modelName = "gemini-3.8-flash";
    [Range(1, 120)]
    public int requestTimeoutSeconds = 30;
    public bool applyFallbackWhenGeminiFails = false;

    [Header("Prompt UI")]
    public TMP_InputField promptInput;
    public Button generatePromptButton;

    [TextArea(2, 5)]
    public string fallbackPrompt = "Warm analog bass with fast attack, strong fundamental, few high harmonics and slow release.";

    [Header("Audio UI")]
    public AudioClip referenceAudioClip;
    public Button analyzeAudioButton;
    public bool useGeminiToComplementAudioAnalysis = false;

    [Header("Status")]
    public TextMeshProUGUI statusText;
    public bool logDetails = true;

    [Serializable]
    public class HarmonicDescriptor
    {
        public int index = 1;
        public float amplitude = 1f;
        public float phaseRadians = 0f;
    }

    [Serializable]
    public class TimbreDescriptor
    {
        public float brightness = 0.35f;
        public float harmonicity = 0.8f;
        public float noisiness = 0.05f;
        public float warmth = 0.5f;
        public float fundamentalStrength = 1f;
        public HarmonicDescriptor[] harmonics = Array.Empty<HarmonicDescriptor>();
    }

    [Serializable]
    public class ADSRDescriptor
    {
        public float attackMs = 10f;
        public float decayMs = 180f;
        public float sustainMs = 600f;
        public float sustainLevel = 0.65f;
        public float releaseMs = 300f;
    }

    [Serializable]
    public class GeneratedTimbre
    {
        public TimbreDescriptor timbre = new TimbreDescriptor();
        public ADSRDescriptor adsr = new ADSRDescriptor();
        public float[] wavetable = Array.Empty<float>();
        public string source = "Unknown";
        public bool usedGemini;
        public bool usedLocalDsp;
        public string message = string.Empty;

        public bool IsUsable()
        {
            return wavetable != null && wavetable.Length == WavetableSize && adsr != null;
        }
    }
}
```



```

    }

    [Serializable]
    private class GeminiStructuredTimbre
    {
        public TimbreDescriptor timbre = new TimbreDescriptor();
        public ADSRDescriptor adsr = new ADSRDescriptor();
        public HarmonicDescriptor[] harmonics = Array.Empty<HarmonicDescriptor>();
    }

#pragma warning disable 0649
    [Serializable]
    private class GeminiGenerateContentResponse
    {
        public GeminiCandidate[] candidates;
        public GeminiError error;
    }

    [Serializable]
    private class GeminiInteractionResponse
    {
        public GeminiInteraction interaction;
        public string outputText;
        public string output_text;
        public GeminiError error;
    }

    [Serializable]
    private class GeminiInteraction
    {
        public string outputText;
        public string output_text;
    }

    [Serializable]
    private class GeminiCandidate
    {
        public GeminiContent content;
        public string finishReason;
    }

    [Serializable]
    private class GeminiContent
    {
        public GeminiPart[] parts;
        public string role;
    }

    [Serializable]
    private class GeminiPart
    {
        public string text;
    }

    [Serializable]
    private class GeminiError
    {
        public int code;
        public string message;
        public string status;
    }
#pragma warning restore 0649

    private Coroutine runningRequest;
    private GeneratedTimbre lastValidTimbre;

    private void Awake()
    {
        if (targetOsc == null)
            targetOsc = FindAnyObjectByType<Osc>();
    }

    private void OnEnable()
    {
        if (generatePromptButton != null)
            generatePromptButton.onClick.AddListener(GenerateFromPromptUI);

        if (analyzeAudioButton != null)
            analyzeAudioButton.onClick.AddListener(GenerateFromAudioUI);
    }

    private void OnDisable()
    {
        if (generatePromptButton != null)
            generatePromptButton.onClick.RemoveListener(GenerateFromPromptUI);

        if (analyzeAudioButton != null)
            analyzeAudioButton.onClick.RemoveListener(GenerateFromAudioUI);
    }

    [ContextMenu("Generate From Prompt")]
    public void GenerateFromPromptUI()
    {
        string prompt = promptInput != null ? promptInput.text : fallbackPrompt;
        GenerateFromPrompt(prompt);
    }

```

```

}

[ContextMenu("Generate From AudioClip")]
public void GenerateFromAudioUI()
{
    GenerateFromAudio(referenceAudioClip);
}

public void GenerateFromPrompt(string prompt)
{
    if (runningRequest != null)
        StopCoroutine(runningRequest);

    runningRequest = StartCoroutine(GenerateFromPromptCoroutine(prompt));
}

[ContextMenu("Generate Local Test Timbre")]
public void GenerateLocalTestTimbre()
{
    string prompt = promptInput != null && !string.IsNullOrEmpty(promptInput.text) ? promptInput.text : fallbackPrompt;
    GeneratedTimbre generated = BuildFallbackTimbre(prompt, "Generado localmente para probar descriptor, wavetable y ADSR sin Gemini.");
    ApplyGeneratedTimbre(generated, null);
}

public IEnumerator GenerateFromPromptCoroutine(string prompt, Action<GeneratedTimbre> onGenerated = null)
{
    if (string.IsNullOrEmpty(prompt))
    {
        ReportFailure("Prompt vacío. No se generó un timbre nuevo.");
        yield break;
    }

    string apiKey = string.IsNullOrEmpty(geminiApiKey) ? string.Empty : geminiApiKey.Trim();
    if (string.IsNullOrEmpty(apiKey))
    {
        ReportFailure("Configura geminiApiKey en el Inspector antes de llamar a Gemini.");

        if (applyFallbackWhenGeminiFails)
            ApplyGeneratedTimbre(BuildFallbackTimbre(prompt, "Fallback local: falta geminiApiKey."), onGenerated);

        yield break;
    }

    SetStatus("Solicitando descriptor timbrico a Gemini...");

    using (UnityWebRequest request = BuildGeminiRequest(prompt, apiKey))
    {
        yield return request.SendWebRequest();

        if (request.result != UnityWebRequest.Result.Success)
        {
            string error = BuildHttpErrorMessage(request);
            ReportFailure(error);

            if (applyFallbackWhenGeminiFails)
                ApplyGeneratedTimbre(BuildFallbackTimbre(prompt, error), onGenerated);

            yield break;
        }

        if (!TryParseGeminiResponse(request.downloadHandler.text, out TimbreDescriptor descriptor, out ADSRDescriptor adsr, out string parseError))
        {
            ReportFailure(parseError);

            if (applyFallbackWhenGeminiFails)
                ApplyGeneratedTimbre(BuildFallbackTimbre(prompt, parseError), onGenerated);

            yield break;
        }

        GeneratedTimbre generated = BuildGeneratedTimbre(descriptor, adsr, "Prompt", true, false, "Generado desde prompt con Gemini.");
        ApplyGeneratedTimbre(generated, onGenerated);
    }
}

public GeneratedTimbre GenerateFromAudio(AudioClip clip)
{
    if (clip == null)
    {
        ReportFailure("AudioClip inválido. Asigna un AudioClip antes de analizar.");
        return null;
    }

    SetStatus("Analizando AudioClip localmente...");

    if (!TryAnalyzeAudioClip(clip, out GeneratedTimbre generated, out string error))
    {
        ReportFailure(error);
        return null;
    }
}

```

```

    }

    ApplyGeneratedTimbre(generated, null);

    if (useGeminiToComplementAudioAnalysis)
        StartCoroutine(ComplementAudioAnalysisWithGemini(generated));

    return generated;
}

public GeneratedTimbre BuildGeneratedTimbre(TimbreDescriptor descriptor, ADSRDescriptor adsr, string source, bool usedGemini, bool usedLocalDsp, string message)
{
    descriptor = descriptor ?? new TimbreDescriptor();
    adsr = adsr ?? new ADSRDescriptor();

    ValidateDescriptor(descriptor);
    ValidateAdsr(adsr);

    return new GeneratedTimbre
    {
        timbre = descriptor,
        adsr = adsr,
        wavetable = BuildWavetableFromDescriptor(descriptor),
        source = source,
        usedGemini = usedGemini,
        usedLocalDsp = usedLocalDsp,
        message = message
    };
}

public static float[] BuildWavetableFromDescriptor(TimbreDescriptor descriptor)
{
    descriptor = descriptor ?? new TimbreDescriptor();
    ValidateDescriptor(descriptor);

    float[] wavetable = new float[WavetableSize];
    HarmonicDescriptor[] harmonics = descriptor.harmonics;

    for (int n = 0; n < WavetableSize; n++)
    {
        float phase01 = (float)n / WavetableSize;
        float value = 0f;

        for (int i = 0; i < harmonics.Length; i++)
        {
            HarmonicDescriptor harmonic = harmonics[i];
            int index = Mathf.Max(1, harmonic.index);
            float amplitude = Mathf.Clamp01(harmonic.amplitude);
            float phase = harmonic.phaseRadians;

            value += amplitude * Mathf.Sin((2f * Mathf.PI * index * phase01) + phase);
        }

        wavetable[n] = value;
    }

    NormalizeInPlace(wavetable);
    return wavetable;
}

private UnityWebRequest BuildGeminiRequest(string prompt, string apiKey)
{
    string body = BuildGeminiRequestJson(prompt);
    byte[] bodyBytes = System.Text.Encoding.UTF8.GetBytes(body);

    UnityWebRequest request = new UnityWebRequest(GeminiEndpoint, UnityWebRequest.kHttpVerbPOST);
    request.uploadHandler = new UploadHandlerRaw(bodyBytes);
    request.downloadHandler = new DownloadHandlerBuffer();
    request.timeout = Mathf.Max(1, requestTimeoutSeconds);
    request.SetRequestHeader("Content-Type", "application/json");
    request.SetRequestHeader("x-goog-api-key", apiKey);
    return request;
}

private string BuildGeminiRequestJson(string prompt)
{
    string instructionalPrompt =
        "You are helping a Unity procedural synthesizer create a wavetable and ADSR envelope. " +
        "Return only structured JSON that matches the schema. " +
        "Do not generate audio samples. Do not return 2048 wavetable values. " +
        "Describe the sound using harmonic amplitudes, phases, timbre values, and ADSR values. " +
        "ADSR units must be milliseconds for attackMs, decayMs, sustainMs, releaseMs, and 0..1 for sustainLevel. " +
        "Keep harmonics musically useful and compact, preferably 1 to 24 harmonics. " +
        "User sound description: " + prompt;

    return "{"
        + "\"model\": \"" + JsonEscape(modelName) + "\", "
        + "\"input\": \"" + JsonEscape(instructionalPrompt) + "\", "
        + "\"response_format\": {"
        + "\"type\": \"text\", "
        + "\"mime_type\": \"application/json\", "
        + "\"schema\": " + BuildResponseSchemaJson()

```

```

        + "}";
        + "}";
    }

private static string BuildResponseSchemaJson()
{
    return "{"
        + "\"type\": \"object\", \""
        + "\"properties\": {"
        + "\"timbre\": {\"type\": \"object\", \"properties\": {"
        + "\"brightness\": {\"type\": \"number\", \"minimum\": 0, \"maximum\": 1}, \""
        + "\"harmonicity\": {\"type\": \"number\", \"minimum\": 0, \"maximum\": 1}, \""
        + "\"noisiness\": {\"type\": \"number\", \"minimum\": 0, \"maximum\": 1}, \""
        + "\"warmth\": {\"type\": \"number\", \"minimum\": 0, \"maximum\": 1}, \""
        + "\"fundamentalStrength\": {\"type\": \"number\", \"minimum\": 0, \"maximum\": 1} \""
        + "}, \"required\": [\"brightness\", \"harmonicity\", \"noisiness\", \"warmth\", \"fundamentalStrength\"]}, \""
        + "\"harmonics\": {\"type\": \"array\", \"minItems\": 1, \"maxItems\": 32, \"items\": {\"type\": \"object\", \"properties\": {"
        + "\"index\": {\"type\": \"integer\", \"minimum\": 1, \"maximum\": 64}, \""
        + "\"amplitude\": {\"type\": \"number\", \"minimum\": 0, \"maximum\": 1}, \""
        + "\"phaseRadians\": {\"type\": \"number\", \"minimum\": -6.283185, \"maximum\": 6.283185} \""
        + "}, \"required\": [\"index\", \"amplitude\", \"phaseRadians\"]}}, \""
        + "\"adsr\": {\"type\": \"object\", \"properties\": {"
        + "\"attackMs\": {\"type\": \"number\", \"minimum\": 1, \"maximum\": 400}, \""
        + "\"decayMs\": {\"type\": \"number\", \"minimum\": 1, \"maximum\": 1000}, \""
        + "\"sustainMs\": {\"type\": \"number\", \"minimum\": 0, \"maximum\": 5000}, \""
        + "\"sustainLevel\": {\"type\": \"number\", \"minimum\": 0.001, \"maximum\": 1}, \""
        + "\"releaseMs\": {\"type\": \"number\", \"minimum\": 1, \"maximum\": 1000} \""
        + "}, \"required\": [\"attackMs\", \"decayMs\", \"sustainMs\", \"sustainLevel\", \"releaseMs\"]} \""
        + "}, \"required\": [\"timbre\", \"harmonics\", \"adsr\"] \""
        + "}"
    }

private bool TryParseGeminiResponse(string responseJson, out TimbreDescriptor descriptor, out ADSRDescriptor adsr, out string error)
{
    descriptor = null;
    adsr = null;
    error = string.Empty;

    if (string.IsNullOrEmpty(responseJson))
    {
        error = "Gemini devolvió una respuesta vacía.";
        return false;
    }

    GeminiInteractionResponse interactionResponse;
    try
    {
        interactionResponse = JsonUtility.FromJson<GeminiInteractionResponse>(responseJson);
    }
    catch (Exception exception)
    {
        error = "No se pudo interpretar la respuesta HTTP de Gemini: " + exception.Message;
        return false;
    }

    if (interactionResponse != null && interactionResponse.error != null && !string.IsNullOrEmpty(interactionResponse.error.message))
    {
        error = "Gemini reportó error " + interactionResponse.error.code + ": " + interactionResponse.error.message;
        return false;
    }

    string structuredJson = ExtractInteractionText(interactionResponse);

    if (string.IsNullOrEmpty(structuredJson))
        structuredJson = ExtractGenerateContentText(responseJson, out error);

    if (string.IsNullOrEmpty(structuredJson) && LooksLikeStructuredTimbreJson(responseJson))
        structuredJson = responseJson;

    if (string.IsNullOrEmpty(structuredJson))
    {
        if (string.IsNullOrEmpty(error))
            error = "Gemini no devolvió texto estructurado utilizable.";

        return false;
    }

    return TryParseStructuredTimbreJson(structuredJson, out descriptor, out adsr, out error);
}

private bool TryParseStructuredTimbreJson(string structuredJson, out TimbreDescriptor descriptor, out ADSRDescriptor adsr, out string error)
{
    descriptor = null;
    adsr = null;
    error = string.Empty;

    if (!LooksLikeStructuredTimbreJson(structuredJson))
    {
        error = "Gemini no devolvió el contrato JSON esperado para timbre, armónicos y ADSR.";
        return false;
    }

    GeminiStructuredTimbre structured;
    try

```

```

    {
        structured = JsonUtility.FromJson<GeminiStructuredTimbre>(structuredJson);
    }
    catch (Exception exception)
    {
        error = "Gemini devolvió JSON inválido: " + exception.Message;
        return false;
    }

    if (structured == null)
    {
        error = "Gemini devolvió JSON vacío.";
        return false;
    }

    descriptor = structured.timbre ?? new TimbreDescriptor();
    if (structured.harmonics == null || structured.harmonics.Length == 0)
    {
        error = "Gemini devolvió un descriptor sin armónicos.";
        return false;
    }

    descriptor.harmonics = structured.harmonics;
    adsr = structured.adsr ?? new ADSRDescriptor();

    ValidateDescriptor(descriptor);
    ValidateAdsr(adsr);
    return true;
}

private static bool LooksLikeStructuredTimbreJson(string json)
{
    if (string.IsNullOrEmpty(json))
        return false;

    return json.Contains("\"timbre\"") &&
           json.Contains("\"harmonics\"") &&
           json.Contains("\"adsr\"");
}

private static string ExtractInteractionText(GeminiInteractionResponse response)
{
    if (response == null)
        return string.Empty;

    if (!string.IsNullOrEmpty(response.outputText))
        return response.outputText;

    if (!string.IsNullOrEmpty(response.output_text))
        return response.output_text;

    if (response.interaction == null)
        return string.Empty;

    if (!string.IsNullOrEmpty(response.interaction.outputText))
        return response.interaction.outputText;

    return response.interaction.output_text ?? string.Empty;
}

private static string ExtractGenerateContentText(string responseJson, out string error)
{
    error = string.Empty;

    GeminiGenerateContentResponse response;
    try
    {
        response = JsonUtility.FromJson<GeminiGenerateContentResponse>(responseJson);
    }
    catch (Exception exception)
    {
        error = "No se pudo interpretar una respuesta legacy de Gemini: " + exception.Message;
        return string.Empty;
    }

    if (response != null && response.error != null && !string.IsNullOrEmpty(response.error.message))
    {
        error = "Gemini reportó error " + response.error.code + ": " + response.error.message;
        return string.Empty;
    }

    if (response == null || response.candidates == null || response.candidates.Length == 0)
        return string.Empty;

    GeminiContent content = response.candidates[0].content;
    if (content == null || content.parts == null || content.parts.Length == 0)
        return string.Empty;

    return content.parts[0].text ?? string.Empty;
}

```

```

private GeneratedTimbre BuildFallbackTimbre(string prompt, string message)
{
    string lower = prompt.ToLowerInvariant();
    float brightness = lower.Contains("bright") || lower.Contains("metallic") ? 0.8f : 0.35f;
    float warmth = lower.Contains("warm") || lower.Contains("analog") ? 0.8f : 0.45f;
    float noisiness = lower.Contains("noise") || lower.Contains("distorted") ? 0.25f : 0.04f;
    float fundamental = lower.Contains("bass") || lower.Contains("fundamental") ? 1f : 0.75f;

    TimbreDescriptor descriptor = new TimbreDescriptor
    {
        brightness = brightness,
        harmonicity = 0.85f,
        noisiness = noisiness,
        warmth = warmth,
        fundamentalStrength = fundamental,
        harmonics = BuildDefaultHarmonics(brightness, warmth, fundamental)
    };

    ADSRDescriptor adsr = new ADSRDescriptor
    {
        attackMs = lower.Contains("slow attack") || lower.Contains("pad") ? 220f : 10f,
        decayMs = 180f,
        sustainMs = lower.Contains("pad") ? 1600f : 650f,
        sustainLevel = lower.Contains("pluck") ? 0.3f : 0.68f,
        releaseMs = lower.Contains("slow release") || lower.Contains("pad") ? 850f : 240f
    };

    return BuildGeneratedTimbre(descriptor, adsr, "PromptFallback", false, true, message);
}

private static HarmonicDescriptor[] BuildDefaultHarmonics(float brightness, float warmth, float fundamentalStrength)
{
    int count = Mathf.RoundToInt(Mathf.Lerp(6f, 18f, Mathf.Clamp01(brightness)));
    HarmonicDescriptor[] harmonics = new HarmonicDescriptor[count];

    for (int i = 0; i < count; i++)
    {
        int harmonicIndex = i + 1;
        float slope = Mathf.Lerp(1.8f, 0.75f, Mathf.Clamp01(brightness));
        float amplitude = Mathf.Pow(1f / harmonicIndex, slope);

        if (harmonicIndex == 1)
            amplitude *= Mathf.Lerp(0.7f, 1.25f, Mathf.Clamp01(fundamentalStrength));

        if (warmth > 0.55f && harmonicIndex % 2 == 0)
            amplitude *= 0.65f;

        harmonics[i] = new HarmonicDescriptor
        {
            index = harmonicIndex,
            amplitude = Mathf.Clamp01(amplitude),
            phaseRadians = 0f
        };
    }

    return harmonics;
}

private bool TryAnalyzeAudioClip(AudioClip clip, out GeneratedTimbre generated, out string error)
{
    generated = null;
    error = string.Empty;

    if (clip.samples <= 0 || clip.channels <= 0)
    {
        error = "El AudioClip no contiene muestras válidas.";
        return false;
    }

    float[] raw = new float[clip.samples * clip.channels];
    if (!clip.GetData(raw, 0))
    {
        error = "No se pudo leer el AudioClip. Revisa que el clip permita cargar sus datos.";
        return false;
    }

    float[] mono = ConvertToMonoAndNormalize(raw, clip.channels);
    if (mono.Length == 0)
    {
        error = "No se pudieron convertir muestras del AudioClip a mono.";
        return false;
    }

    float[] envelope = BuildSmoothedEnvelope(mono, clip.frequency);
    ADSRDescriptor adsr = EstimateAdsrFromEnvelope(envelope, clip.frequency);

    int period = EstimatePeriodByAutocorrelation(mono, envelope, clip.frequency);
    float[] wavetable = period > 0
        ? ExtractCycleAsWavetable(mono, envelope, period)
        : ResampleWindowAsWavetable(mono, FindStableStart(envelope));

    TimbreDescriptor descriptor = EstimateDescriptorFromAudio(wavetable, adsr);
}

```

```

        generated = new GeneratedTimbre
        {
            timbre = descriptor,
            adsr = adsr,
            wavetable = wavetable,
            source = "AudioClip",
            usedGemini = false,
            usedLocalDsp = true,
            message = "Generado desde AudioClip con DSP local."
        };

        return generated.IsUsable();
    }

    private static float[] ConvertToMonoAndNormalize(float[] raw, int channels)
    {
        int safeChannels = Mathf.Max(1, channels);
        int frames = raw.Length / safeChannels;
        float[] mono = new float[frames];
        float maxAbs = 0f;

        for (int frame = 0; frame < frames; frame++)
        {
            float sum = 0f;
            int baseIndex = frame * safeChannels;

            for (int channel = 0; channel < safeChannels; channel++)
                sum += raw[baseIndex + channel];

            float value = sum / safeChannels;
            mono[frame] = value;
            maxAbs = Mathf.Max(maxAbs, Mathf.Abs(value));
        }

        if (maxAbs > 0.000001f)
        {
            for (int i = 0; i < mono.Length; i++)
                mono[i] /= maxAbs;
        }

        return mono;
    }

    private static float[] BuildSmoothedEnvelope(float[] mono, int sampleRate)
    {
        float[] envelope = new float[mono.Length];
        int window = Mathf.Clamp(sampleRate / 200, 16, 512);
        float sum = 0f;

        for (int i = 0; i < mono.Length; i++)
        {
            sum += Mathf.Abs(mono[i]);

            if (i >= window)
                sum -= Mathf.Abs(mono[i - window]);

            int count = Mathf.Min(i + 1, window);
            envelope[i] = sum / Mathf.Max(1, count);
        }

        NormalizeInPlace(envelope, true);
        return envelope;
    }

    private static ADSRDescriptor EstimateAdsrFromEnvelope(float[] envelope, int sampleRate)
    {
        int peakIndex = 0;
        float peak = 0f;

        for (int i = 0; i < envelope.Length; i++)
        {
            if (envelope[i] > peak)
            {
                peak = envelope[i];
                peakIndex = i;
            }
        }

        if (peak <= 0.0001f)
            return new ADSRDescriptor();

        int onset = FindFirstAbove(envelope, peak * 0.08f, 0, envelope.Length - 1);
        int attackEnd = FindFirstAbove(envelope, peak * 0.9f, onset, Mathf.Max(onset, peakIndex));
        int end = FindLastAbove(envelope, peak * 0.08f);

        float sustainLevel = EstimateSustainLevel(envelope, peakIndex, end);
        int decayEnd = FindFirstBelow(envelope, sustainLevel + ((peak - sustainLevel) * 0.15f), peakIndex, end);
        int releaseStart = FindReleaseStart(envelope, sustainLevel, peakIndex, end);

        float attackMs = FramesToMs(Mathf.Max(1, attackEnd - onset), sampleRate);
        float decayMs = FramesToMs(Mathf.Max(1, decayEnd - attackEnd), sampleRate);
        float sustainMs = FramesToMs(Mathf.Max(0, releaseStart - decayEnd), sampleRate);
    }

```

```

float releaseMs = FramesToMs(Mathf.Max(1, end - releaseStart), sampleRate);

ADSRDescriptor adsr = new ADSRDescriptor
{
    attackMs = attackMs,
    decayMs = decayMs,
    sustainMs = sustainMs,
    sustainLevel = sustainLevel,
    releaseMs = releaseMs
};

ValidateAdsr(adsr);
return adsr;
}

private static int EstimatePeriodByAutocorrelation(float[] mono, float[] envelope, int sampleRate)
{
    int start = FindStableStart(envelope);
    int windowSize = Mathf.Min(4096, mono.Length - start);

    if (windowSize < 512)
        return -1;

    int minLag = Mathf.Max(1, sampleRate / 1200);
    int maxLag = Mathf.Min(sampleRate / 50, windowSize / 2);
    float bestScore = 0f;
    int bestLag = -1;

    for (int lag = minLag; lag <= maxLag; lag++)
    {
        float sum = 0f;
        float aEnergy = 0f;
        float bEnergy = 0f;
        int count = windowSize - lag;

        for (int i = 0; i < count; i++)
        {
            float a = mono[start + i];
            float b = mono[start + i + lag];
            sum += a * b;
            aEnergy += a * a;
            bEnergy += b * b;
        }

        float denom = Mathf.Sqrt(aEnergy * bEnergy);
        if (denom <= 0.000001f)
            continue;

        float score = sum / denom;
        if (score > bestScore)
        {
            bestScore = score;
            bestLag = lag;
        }
    }

    return bestScore >= 0.35f ? bestLag : -1;
}

private static float[] ExtractCycleAsWavetable(float[] mono, float[] envelope, int period)
{
    int stableStart = FindStableStart(envelope);
    int start = FindNearestRisingZeroCrossing(mono, stableStart, Mathf.Min(mono.Length - 1, stableStart + period));
    int end = Mathf.Min(mono.Length - 1, start + Mathf.Max(1, period));

    if (end <= start + 1)
        return ResampleWindowAsWavetable(mono, stableStart);

    float[] wavetable = new float[WavetableSize];
    int length = end - start;

    for (int i = 0; i < WavetableSize; i++)
    {
        float readPosition = start + ((float)i / WavetableSize) * length;
        int i0 = Mathf.Clamp(Mathf.FloorToInt(readPosition), 0, mono.Length - 1);
        int i1 = Mathf.Clamp(i0 + 1, 0, mono.Length - 1);
        float frac = readPosition - i0;
        wavetable[i] = Mathf.Lerp(mono[i0], mono[i1], frac);
    }

    RemoveDcAndNormalize(wavetable);
    return wavetable;
}

private static float[] ResampleWindowAsWavetable(float[] mono, int start)
{
    float[] wavetable = new float[WavetableSize];
    int safeStart = Mathf.Clamp(start, 0, Mathf.Max(0, mono.Length - 2));
    int available = Mathf.Max(1, mono.Length - safeStart - 1);
    int length = Mathf.Min(available, WavetableSize);

    for (int i = 0; i < WavetableSize; i++)
    {

```



```

        float readPosition = safeStart + ((float)i / WavetableSize) * length;
        int i0 = Mathf.Clamp(Mathf.FloorToInt(readPosition), 0, mono.Length - 1);
        int i1 = Mathf.Clamp(i0 + 1, 0, mono.Length - 1);
        float frac = readPosition - i0;
        wavetable[i] = Mathf.Lerp(mono[i0], mono[i1], frac);
    }

    RemoveDcAndNormalize(wavetable);
    return wavetable;
}

private static TimbreDescriptor EstimateDescriptorFromAudio(float[] wavetable, ADSRDescriptor adsr)
{
    float roughness = 0f;
    for (int i = 1; i < wavetable.Length; i++)
        roughness += Mathf.Abs(wavetable[i] - wavetable[i - 1]);

    roughness /= Mathf.Max(1, wavetable.Length - 1);

    float brightness = Mathf.Clamp01(roughness * 2.2f);
    float warmth = Mathf.Clamp01(1f - (brightness * 0.7f));
    float attackFastness = 1f - Mathf.Clamp01(adsr.attackMs / 400f);

    return new TimbreDescriptor
    {
        brightness = brightness,
        harmonicity = 0.7f,
        noisiness = Mathf.Clamp01(roughness * 0.6f),
        warmth = warmth,
        fundamentalStrength = Mathf.Lerp(0.7f, 1f, attackFastness),
        harmonics = BuildDefaultHarmonics(brightness, warmth, 1f)
    };
}

private IEnumerator ComplementAudioAnalysisWithGemini(GeneratedTimbre generated)
{
    if (generated == null)
        yield break;

    string prompt = "Analyze this locally extracted audio descriptor and return a compact timbre descriptor. "
        + "Do not ask for audio samples. Local DSP already produced the wavetable. "
        + "Use the ADSR and rough timbre values to refine harmonics only. "
        + "Local values: brightness=" + generated.timbre.brightness.ToString("0.###", CultureInfo.InvariantCulture)
        + ", noisiness=" + generated.timbre.noisiness.ToString("0.###", CultureInfo.InvariantCulture)
        + ", attackMs=" + generated.adsr.attackMs.ToString("0.###", CultureInfo.InvariantCulture)
        + ", releaseMs=" + generated.adsr.releaseMs.ToString("0.###", CultureInfo.InvariantCulture) + ".";

    yield return GenerateFromPromptCoroutine(prompt);
}

private void ApplyGeneratedTimbre(GeneratedTimbre generated, Action<GeneratedTimbre> onGenerated)
{
    if (generated == null || !generated.IsUsable())
    {
        ReportFailure("El timbre generado no es utilizable.");
        return;
    }

    if (targetOsc == null)
        targetOsc = FindAnyObjectByType<Osc>();

    if (targetOsc != null)
        targetOsc.ApplyGeneratedTimbre(generated);

    lastValidTimbre = generated;
    onGenerated?.Invoke(generated);
    SetStatus(generated.message);

    if (logDetails)
        Debug.Log("[GeminiTimbreGenerator] " + generated.message);
}

private void ReportFailure(string message)
{
    SetStatus(message);
    Debug.LogWarning("[GeminiTimbreGenerator] " + message);
}

private void SetStatus(string message)
{
    if (statusText != null)
        statusText.SetText(message);
}

private static string BuildHttpErrorMessage(UnityWebRequest request)
{
    long code = request.responseCode;
    string prefix;

    if (code == 401 || code == 403)
        prefix = "Gemini rechazó la credencial o los permisos.";
    else if (code == 429)
        prefix = "Gemini reportó límite de cuota o rate limit.";
}

```

```

        else if (code >= 500)
            prefix = "Gemini no está disponible temporalmente.";
        else if (request.result == UnityWebRequest.Result.ConnectionError)
            prefix = "No hay conexión con Gemini.";
        else if (request.result == UnityWebRequest.Result.ProtocolError)
            prefix = "Gemini devolvió un error HTTP.";
        else if (request.result == UnityWebRequest.Result.DataProcessingError)
            prefix = "No se pudo procesar la respuesta de Gemini.";
        else
            prefix = "La solicitud a Gemini falló.";

        return prefix + " HTTP " + code + ". " + request.error;
    }

    private static void ValidateDescriptor(TimbreDescriptor descriptor)
    {
        if (descriptor == null)
            return;

        descriptor.brightness = Mathf.Clamp01(descriptor.brightness);
        descriptor.harmonicity = Mathf.Clamp01(descriptor.harmonicity);
        descriptor.noisiness = Mathf.Clamp01(descriptor.noisiness);
        descriptor.warmth = Mathf.Clamp01(descriptor.warmth);
        descriptor.fundamentalStrength = Mathf.Clamp01(descriptor.fundamentalStrength);

        if (descriptor.harmonics == null || descriptor.harmonics.Length == 0)
            descriptor.harmonics = BuildDefaultHarmonics(descriptor.brightness, descriptor.warmth, descriptor.fundamentalStrength);

        int count = Mathf.Clamp(descriptor.harmonics.Length, 1, 64);
        if (count != descriptor.harmonics.Length)
            Array.Resize(ref descriptor.harmonics, count);

        for (int i = 0; i < descriptor.harmonics.Length; i++)
        {
            if (descriptor.harmonics[i] == null)
                descriptor.harmonics[i] = new HarmonicDescriptor { index = i + 1, amplitude = 0f, phaseRadians = 0f };

            descriptor.harmonics[i].index = Mathf.Clamp(descriptor.harmonics[i].index, 1, 128);
            descriptor.harmonics[i].amplitude = Mathf.Clamp01(descriptor.harmonics[i].amplitude);
            descriptor.harmonics[i].phaseRadians = Mathf.Clamp(descriptor.harmonics[i].phaseRadians, -Mathf.PI * 2f, Mathf.PI * 2f);
        }
    }

    private static void ValidateAdsr(ADSRDescriptor adsr)
    {
        if (adsr == null)
            return;

        adsr.attackMs = Mathf.Clamp(adsr.attackMs, 1f, 400f);
        adsr.decayMs = Mathf.Clamp(adsr.decayMs, 1f, 1000f);
        adsr.sustainMs = Mathf.Clamp(adsr.sustainMs, 0f, 5000f);
        adsr.sustainLevel = Mathf.Clamp(adsr.sustainLevel, 0.001f, 1f);
        adsr.releaseMs = Mathf.Clamp(adsr.releaseMs, 1f, 1000f);
    }

    private static void NormalizeInPlace(float[] data, bool positiveOnly = false)
    {
        if (data == null || data.Length == 0)
            return;

        float max = 0f;
        for (int i = 0; i < data.Length; i++)
            max = Mathf.Max(max, positiveOnly ? data[i] : Mathf.Abs(data[i]));

        if (max <= 0.000001f)
            return;

        for (int i = 0; i < data.Length; i++)
            data[i] = Mathf.Clamp(data[i] / max, positiveOnly ? 0f : -1f, 1f);
    }

    private static void RemoveDcAndNormalize(float[] data)
    {
        if (data == null || data.Length == 0)
            return;

        float mean = 0f;
        for (int i = 0; i < data.Length; i++)
            mean += data[i];

        mean /= data.Length;

        for (int i = 0; i < data.Length; i++)
            data[i] -= mean;

        NormalizeInPlace(data);
    }

    private static int FindStableStart(float[] envelope)
    {
        if (envelope == null || envelope.Length == 0)

```

```

        return 0;

    int peakIndex = 0;
    float peak = 0f;

    for (int i = 0; i < envelope.Length; i++)
    {
        if (envelope[i] > peak)
        {
            peak = envelope[i];
            peakIndex = i;
        }
    }

    return Mathf.Clamp(peakIndex + Mathf.Max(32, envelope.Length / 50), 0, Mathf.Max(0, envelope.Length - 2));
}

private static int FindNearestRisingZeroCrossing(float[] mono, int start, int end)
{
    int safeStart = Mathf.Clamp(start, 1, Mathf.Max(1, mono.Length - 1));
    int safeEnd = Mathf.Clamp(end, safeStart, mono.Length - 1);

    for (int i = safeStart; i <= safeEnd; i++)
    {
        if (mono[i - 1] <= 0f && mono[i] > 0f)
            return i;
    }

    return safeStart;
}

private static int FindFirstAbove(float[] data, float threshold, int start, int end)
{
    int safeStart = Mathf.Clamp(start, 0, Mathf.Max(0, data.Length - 1));
    int safeEnd = Mathf.Clamp(end, safeStart, Mathf.Max(0, data.Length - 1));

    for (int i = safeStart; i <= safeEnd; i++)
    {
        if (data[i] >= threshold)
            return i;
    }

    return safeStart;
}

private static int FindFirstBelow(float[] data, float threshold, int start, int end)
{
    int safeStart = Mathf.Clamp(start, 0, Mathf.Max(0, data.Length - 1));
    int safeEnd = Mathf.Clamp(end, safeStart, Mathf.Max(0, data.Length - 1));

    for (int i = safeStart; i <= safeEnd; i++)
    {
        if (data[i] <= threshold)
            return i;
    }

    return safeEnd;
}

private static int FindLastAbove(float[] data, float threshold)
{
    for (int i = data.Length - 1; i >= 0; i--)
    {
        if (data[i] >= threshold)
            return i;
    }

    return Mathf.Max(0, data.Length - 1);
}

private static float EstimateSustainLevel(float[] envelope, int peakIndex, int end)
{
    int start = Mathf.Clamp(peakIndex + ((end - peakIndex) / 2), 0, Mathf.Max(0, envelope.Length - 1));
    int safeEnd = Mathf.Clamp(end, start, Mathf.Max(0, envelope.Length - 1));
    float sum = 0f;
    int count = 0;

    for (int i = start; i <= safeEnd; i++)
    {
        sum += envelope[i];
        count++;
    }

    if (count == 0)
        return 0.5f;

    return Mathf.Clamp(sum / count, 0.001f, 1f);
}

private static int FindReleaseStart(float[] envelope, float sustainLevel, int peakIndex, int end)
{

```

```

float threshold = Mathf.Max(0.02f, sustainLevel * 0.75f);

for (int i = Mathf.Max(peakIndex, 0); i <= end; i++)
{
    if (envelope[i] < threshold)
        return i;
}

return Mathf.Clamp(end - Mathf.Max(1, envelope.Length / 10), peakIndex, end);
}

private static float FramesToMs(int frames, int sampleRate)
{
    return (frames / Mathf.Max(1f, sampleRate)) * 1000f;
}

private static string JsonEscape(string value)
{
    if (string.IsNullOrEmpty(value))
        return string.Empty;

    return value
        .Replace("\\", "\\\\")
        .Replace("\"", "\\\"")
        .Replace("\n", "\\n")
        .Replace("\r", "\\r")
        .Replace("\t", "\\t");
}
}

```

## 18.2 Modificaciones necesarias en Osc.cs

```

public void ApplyGeneratedTimbre(GeminiTimbreGenerator.GeneratedTimbre generatedTimbre)
{
    if (generatedTimbre == null || generatedTimbre.wavetable == null || generatedTimbre.wavetable.Length == 0 || generatedTimbre.adsr
== null)
    {
        Debug.LogWarning("[Osc] Generated timbre is not valid. Current synth settings were preserved.");
        return;
    }

    float[] generatedWavetable = new float[wavetableSize];
    int sourceLength = generatedTimbre.wavetable.Length;

    for (int i = 0; i < wavetableSize; i++)
    {
        float readPos = ((float)i / wavetableSize) * sourceLength;
        int index0 = Mathf.FloorToInt(readPos) % sourceLength;
        int index1 = (index0 + 1) % sourceLength;
        float frac = readPos - Mathf.Floor(readPos);
        generatedWavetable[i] = Mathf.Clamp(Mathf.Lerp(generatedTimbre.wavetable[index0], generatedTimbre.wavetable[index1], frac), -
1f, 1f);
    }

    wavetable = generatedWavetable;
    FormType = WaveFormType.Custom1;
    useWavetable = true;
    currentWavetableLoadedFromExternal = true;
    useAudioClipADSR = false;

    A = Mathf.Clamp(generatedTimbre.adsr.attackMs, 1f, 400f);
    D = Mathf.Clamp(generatedTimbre.adsr.decayMs, 1f, 1000f);
    S = Mathf.Clamp(generatedTimbre.adsr.sustainMs, 0f, 5000f);
    SL = Mathf.Clamp(generatedTimbre.adsr.sustainLevel, 0.001f, 1f);
    R = Mathf.Clamp(generatedTimbre.adsr.releaseMs, 1f, 1000f);
    SLegacy = SL;

    UpdateADSR();
    MarkExternalBackendDirty();
}

```

## 19 Actividad final y lista de verificación

Actividad final: genera un bajo oscuro desde prompt y luego genera una wavetable desde un AudioClip tonal. Compara el ADSR y el timbre de ambos resultados.

- API\_KEY\_CREATION\_DOCUMENTED=YES
- API\_KEY\_CONFIGURATION\_DOCUMENTED=YES

- GEMINI\_CLASS\_CREATED\_FROM\_ZERO=YES
- USINGS\_DOCUMENTED=YES
- DATA\_STRUCTURES\_DOCUMENTED=YES
- API\_REQUEST\_CODE\_DOCUMENTED=YES
- STRUCTURED\_OUTPUT\_DOCUMENTED=YES
- JSON\_PARSING\_DOCUMENTED=YES
- VALIDATION\_DOCUMENTED=YES
- WAVETABLE\_CODE\_DOCUMENTED=YES
- ADDITIVE\_SYNTHESIS\_EXPLAINED=YES
- ADSR\_CODE\_DOCUMENTED=YES
- OSC\_MODIFICATION\_CODE\_DOCUMENTED=YES
- AUDIOCLIP\_GETDATA\_DOCUMENTED=YES
- MONO\_CONVERSION\_DOCUMENTED=YES
- NORMALIZATION\_DOCUMENTED=YES
- ENVELOPE\_ANALYSIS\_DOCUMENTED=YES
- PITCH\_PERIOD\_ANALYSIS\_DOCUMENTED=YES
- CYCLE\_EXTRACTION\_DOCUMENTED=YES
- RESAMPLING\_DOCUMENTED=YES
- UNITY\_UI\_SETUP\_DOCUMENTED=YES
- PROMPT\_TEST\_REPRODUCIBLE=YES
- AUDIO\_TEST\_REPRODUCIBLE=YES
- FULL\_GEMINI\_CLASS\_INCLUDED=YES

Pregunta final de validacion: si se elimina GeminiTimbreGenerator.cs y se entrega solo el sintetizador base con este documento, el estudiante puede reconstruir la funcionalidad. Respuesta: YES.

## 20 Referencias oficiales

- Using Gemini API keys. Google AI for Developers. <https://ai.google.dev/gemini-api/docs/api-key>
- Getting started with the Gemini API. Google AI for Developers. <https://ai.google.dev/gemini-api/docs/get-started>
- Structured outputs. Google AI for Developers. <https://ai.google.dev/gemini-api/docs/structured-output>
- Gemini Interactions API reference. Google AI for Developers. <https://ai.google.dev/api/interactions-api-v1>
- Troubleshoot Google AI Studio. Google AI for Developers. <https://ai.google.dev/gemini-api/docs/troubleshoot-ai-studio>